

Module: II

Requirements Analysis and Design

Q1. State the objective of requirement analysis and specification

The aim of requirement analysis and specification phase is to understand the exact requirements of the customer and to document them properly.

Q2. Define SRS

The output of requirement analysis and specification is a document called Software Requirement Specification (SRS) that describes what the proposed software should do without describing how the software will do it.

Q3. What is the role of System Analyst in requirement analysis and specification

The engineers who gather and analyse customer requirements and then write the requirements specification document are known as System Analysts.

Q4. List the tasks involved in requirement analysis and specification

1. Requirement gathering and analysis
2. Requirement specification

Q5. Write notes on Requirements Gathering

Requirement gathering is also known as requirement elicitation. Gathering activity starts by studying the existing documents to collect all possible information about the system. System analyst visits the customer site and interviews the end - users and customers. Normally customers describe their requirements in very vague terms. Good analysts try to extract the requirements as precisely as possible by carrying out intensive discussions with the customer.

Q6. List the different ways used for requirements gathering

1. **Studying the existing documentation** - the analyst usually studies all existing documents about the system before visiting the site.
2. **Interview** - all the different categories of the users are interviewed to gather different functionalities required by them.
3. **Task analysis** - the software provides a set of services (also called tasks). For each task, the analyst tries to formulate the different steps in consultation with the users
4. **Scenario analysis** - for various tasks, the different types of scenarios that may occur and the details of each scenario is identified in consultation with the users.
5. **Form analysis** - the different forms are analysed to determine the data input to the system and the data that are output from the system

Q7. Write notes on requirement analysis

After requirement gathering is complete, the analyst analyses the gathered requirements to clearly understand the exact customer requirements and to weed out any problems in the gathered requirements. The main purpose of the requirements analysis activity is to analyse the collected information to obtain a clear understanding of the product to be developed while removing all ambiguities, incompleteness and inconsistencies from the initial customer perception of the problem

Q8. What are the problems that may be present in the gathered requirements

1. **Anomaly** - an anomaly is an ambiguity in the requirement. Ambiguity refers to the presence of unclear, vague, or multiple interpretations of the information or specifications provided.

Example: In an online shopping cart system, one of the requirement statements may be in the form

“The system should provide a fast checkout process for customers”

Here the term fast is ambiguous. To eliminate the ambiguity, the statement can be rewritten as

“The system should process and confirm an online order within 30 seconds from the time the 'Checkout' button is clicked.”

2. **Inconsistency** - inconsistency refers to a situation where different parts of the collected requirements or specifications contradict each other, leading to confusion or conflicts in understanding.

Example: In a Travel Booking System, there may be two requirements as stated below

Requirement 1: "The system shall allow users to book flights up to 12 hours before departure."

Requirement 2: "Users must complete the flight booking process at least 24 hours before the departure time."

These two requirements statements are contradictory. To eliminate the inconsistency, the statement can be revised as

Revised Requirement: "Users should be able to book flights up to 24 hours before departure."

3. **Incompleteness** - incompleteness refers to a situation where the gathered specifications, descriptions, or information about a product or system are insufficient or lacking in detail.

Example: In an Online Shopping Website, a requirement can be

“Users should be able to add products to their shopping cart”

While this requirement sounds straightforward, it lacks several critical details that could lead to misunderstandings or incorrect implementations such as quantity and variants, user authentication etc

Q9. Write notes on Software Requirement Specification (SRS) document

The requirements identified during the requirements gathering and analysis activity are organised into a document called SRS. The SRS document is written in end-user terminology. The SRS document normally serves as a contract between the development team and the customer. The three most important contents of SRS are

1. Functional requirements
2. Non-functional requirements
3. Goals of implementation

Q10. List the different types of requirements in SRS

Functional requirements: The functional requirements discuss the functionalities required by the users from the system. They should clearly describe each function which the system would support along with the corresponding input and output dataset.

Non-functional requirements: Non-functional requirements address aspects concerning maintainability, portability, usability, maximum number of concurrent users, timing and throughput. It may include reliability issues, accuracy of results, human computer interface issues etc.

Goals of implementation: It offers some general suggestions regarding development. They are not checked by the customer at the time of acceptance testing. It may include issues such as revisions to the system functionalities that may be required in the future, new devices to be supported in the future, reusability etc.

Q11. List the different categories of users who use SRS document

1. **Users, customers and marketing personnel** - refer the SRS document to ensure that the system as described will meet their needs.
2. **Software developers** - refers the SRS document to make sure that they are developing exactly what is required by the customer
3. **Test engineers** - to ensure that the requirements are understandable, so that they can test the software and validate its working
4. **User documentation writers** - to ensure that they understand the features of the product so that they can write the user manuals

5. Project managers - to ensure that they can estimate the cost of the project easily by referring to the SRS and that it contains all information required to plan the project

6. Maintenance engineers - helps the maintenance engineers to understand the functionalities of the system so that they can implement required modifications later

Q12. What are the characteristics of a good SRS document

1. **Correct** – An SRS is correct if every requirement included in the SRS represents something required in the final system.
2. **Complete** – An SRS is complete if everything the software is supposed to do and the responses of the software to all classes of input data are specified in the SRS.
3. **Unambiguous** – An SRS is unambiguous if and only if every requirement stated has one and only one interpretation.
4. **Verifiable** – An SRS is verifiable if and only if every stated requirement is verifiable. A requirement is verifiable if there exists some cost-effective process that can check whether the final software meets that requirement.
5. **Consistent** – An SRS is consistent if there is no requirement that conflicts with another.
6. **Ranked for importance and/or stability** – An SRS is ranked for importance and/or stability if for each requirement the importance and stability of the requirement are indicated.

Q13. Write notes on Software Design

Software design transforms the SRS document into the Design Document. Main outcomes of design process are

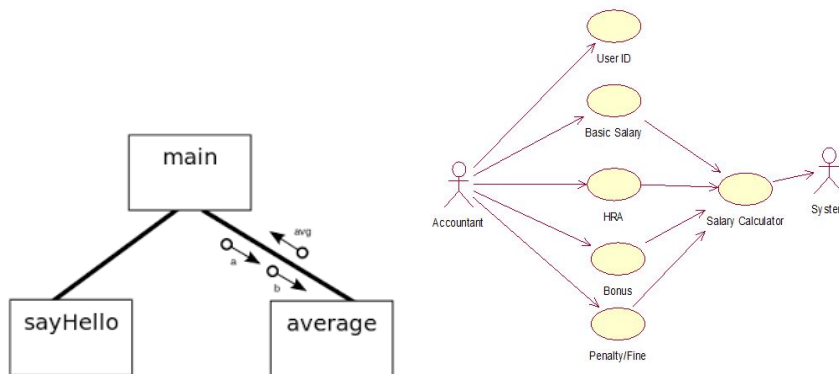
- Different modules required
- Control relationships among modules
- Interfaces among different modules
- Data structures of the individual modules
- Algorithms required to implement the individual modules

Q14. List and explain the different activities in Software Design

1. High-level design - also called preliminary design. A problem is decomposed into a set of modules and control relationships and interfaces among various modules are identified. The outcome of high-level design is called the program structure or the software architecture. Various notations are used to represent a high-level design such as

A tree-like diagram called the structure chart

Unified Modeling Language (UML)



Structure Chart

UML

2. Detailed design - during detailed design, each module is examined carefully to design its data structures and the algorithms. The outcome of the detailed design stage is usually documented in the form of a module specification (MSPEC) document

Q15. List the characteristics of a good design

- 1. Correctness** - a good design should correctly implement all the functionalities of the system.
- 2. Understandability** - a good design should be easily understandable. Otherwise it would be difficult to implement and maintain it.
- 3. Efficiency** - a good design should address resource, time, and cost optimization issues
- 4. Maintainability** - a good design should be easy to change. This is important because change requests usually keep coming from the customer even after product release.

Q16. Explain Cohesion and Coupling

Cohesion is a measure of the functional strength of a module, whereas coupling between two modules is a measure of the degree of interaction between the two modules. High cohesion and low coupling of modules indicates good module decomposition. A module that is highly cohesive and also has low coupling with other modules is said to be functionally independent of the other modules.



Q17. Explain Cohesion and its types

Cohesion of a module represents how tightly the internal elements of the module are bound to one another. The different types of cohesion are

1. **Coincidental Cohesion** - occurs when there is no meaningful relationship among the elements of a module. It is the lowest cohesion.
2. **Logical Cohesion** - a module is said to be logically cohesive, if all elements of the module perform similar operations such as error handling, data input, data output etc.
3. **Temporal Cohesion** - When a module contains functions that are related by the fact that these functions are executed in the same time span, then the module is said to possess temporal cohesion.
4. **Procedural Cohesion** - A module is said to possess procedural cohesion, if the set of functions of the module are executed one after the other, even though these functions perform different purposes and on different data.
5. **Communicational Cohesion** - A module is said to have communicational cohesion, if all functions of the module refer to or update the same data structure.
6. **Sequential Cohesion** - A module is said to possess sequential cohesion, if the different functions of the module are executed in a sequence and the output from one function is input to the next in the sequence.
7. **Functional Cohesion** - A module is said to possess functional cohesion, if different functions of the module cooperate to complete a single task. It is the strongest cohesion.



Q18. Explain coupling and its types

Coupling between modules is the strength of interconnections between modules or a measure of interdependence among modules. The different types of coupling are



1. **Data Coupling:** Two modules are data coupled, if they communicate using an elementary data that is passed as a parameter between the two.

2. Stamp Coupling: Two modules are stamp coupled, if they communicate using a composite data item such as a record in PASCAL or structure in C.

3. Control Coupling: Control coupling exists between two modules, if data from one module is used to direct the order of instruction execution in another. Eg: a flag set in one module and tested in another module

4. Common Coupling: Two modules are common coupled, if they share some global data items

5. Content Coupling: Content coupling exists between two modules, if they share code. For example, a jump from one module into the code of another module can occur.

Q19. Explain the different approaches to Software Design

1. Function-Oriented Design:

In Function-oriented design, a software system is broken down into smaller functional components or modules. Each module is designed to perform a specific function or task. This approach is often associated with procedural programming languages. Function-oriented design is characterised by the following principles:

Modularity: The system is divided into smaller, manageable modules, each responsible for a specific function. This promotes code organisation and maintainability.

Reusability: The modules can be used in different parts of the program or even in other projects.

Encapsulation: Modules encapsulate their logic and data, making them independent and reducing the likelihood of unintended interactions.

Separation of Concerns: Each module performs a distinct functionality, which aids in understanding, debugging, and maintenance.

Hierarchical Structure: Modules can be organised in a hierarchical manner, allowing for a clear representation of dependencies and interactions.

Testing and Debugging: Modules can be individually tested and debugged, simplifying the identification and resolution of issues.

2. Object-Oriented Design: Object-oriented design (OOD) is a design approach that is based on the concept of objects. Objects are instances of classes, which define both data (attributes) and behaviour (methods) that the objects possess. Object-oriented design is associated with languages like Java, C++, Python, and others, and it is characterised by the following principles:

Abstraction: Objects abstract real-world entities by representing their attributes and behaviours.

Encapsulation: Objects encapsulate their data and methods, limiting access and preventing direct manipulation of data.

Inheritance: Classes can inherit attributes and behaviours from other classes, allowing for the creation of hierarchies and the reuse of code.

Polymorphism: Different objects can respond to the same method call in different ways, enabling flexible and extensible code.

Modularity: Classes and objects promote modularity by encapsulating related data and behaviour into self-contained units.

Q20. Define User Interface

The visual part of a computer application or operating system through which a client interacts with a computer or software. It determines how commands are given to the computer or the program and how data is displayed on the screen.

Q21. What are the different types of User Interfaces

1. Text-Based User Interface or Command Line Interface
2. Graphical User Interface (GUI)

Text-Based User Interface: This method relies primarily on the keyboard. A typical example of this is UNIX.

Advantages

- Many and easier customizations options.
- Typically capable of more important tasks.

Disadvantages

- Relies heavily on recall rather than recognition.
- Navigation is often more difficult.

Graphical User Interface (GUI): GUI relies much more heavily on the mouse. A typical example of this type of interface is any versions of the Windows operating systems. The main contents of a GUI are windows, icons, menus etc.

Advantages

- Less expert knowledge is required to use it.
- Easier to Navigate and can look through folders quickly in a guess and check manner.

- The user may switch quickly from one task to another and can interact with several different applications.

Disadvantages

- Typically decreased options.
- Usually less customizable. Not easy to use one button for tons of different variations.

Q22. What are the characteristics of a good user interface design

- 1. Clear** - Meaning and functions of the UI should be clearly communicated to the user
- 2. Concise** - Explanation of the UI functionality should be concise
- 3. Responsive** - The UI should respond fast and provide feedbacks to users
- 4. Consistent** - There should be a fair level of consistency that the interface should maintain throughout its working.
- 5. Attractive** - The look and feel of the UI should be attractive.
- 6. Efficient** - Implement an interface that lets people easily accomplish what they want instead of simply implementing access to a list of features.
- 7. Familiar** - Identify things that are familiar to the users and integrate them into the user interface
- 8. Forgiving** - People are bound to make mistakes when using a website or software. How a software handles those mistakes is an important indicator of the quality of that software.